IETF 127 BoF · ART Area

MTA Hooks

An HTTP protocol that lets an MTA delegate processing decisions to external services, across the whole life of a message.

draft-degennaro-mta-hooks-02
Mauro De Gennaro, Stalwart Labs

One state object
Session, envelope, message and delivery, described once.

One JSON Patch
Every decision is a change to that state.

Grants
The MTA decides what may change.

Nine stages
From connect to the DSN.

Plain HTTP
JSON or CBOR.

Today

MTAs delegate through three kinds of interface

`connect` `helo` `mail` `rcpt` `data` `route` `send` `outcome` `bounce`

Milter | Lookup tables | Logs

Reception

Milter

Filters the SMTP conversation, from connect to the end of DATA, while the client waits.

Routing

Lookup tables

Transport, next hop and address maps: a key in, a value out.

Outcomes

Logs

Deliveries, deferrals and bounces, read from logs and bounce mailboxes.

Each was designed for its own job, and does it well.

Today

Milter is the established reception-time filter

`connect` `helo` `envfrom` `envrcpt` `header` `body` `eom`

Built for reception

Callbacks along the SMTP conversation. The filter can reject, defer, discard or modify while the client waits for the reply.

Widely deployed

Sendmail around 2000, through `libmilter`, and Postfix since. Rspamd, ClamAV, SpamAssassin, OpenDKIM and OpenDMARC all speak it.

Unchanged by this work

Specifying Milter is out of scope. Gateways in either direction are expected to exist.

Beyond reception

The charter adds routing, delivery, deferral and DSNs

Draft charter

“…across the whole lifetime of a message: from the SMTP conversation that receives it, through routing and delivery, to the generation of delivery status notifications.”

Routing

Before delivery, per recipient, including the TLS policy for each destination.

Delivery

The outcome of each attempt, recipient by recipient.

Deferral

Retries, their timing, and when the next notice is due.

DSN

Before the MTA sends a notification it generated.

New capabilities for a broader protocol. Archiving and TLS reporting are examples built on them.

Routing

Routing decisions need the message's full context

An external routing lookup receives

one key

The recipient domain or address. A transport and next hop come back.

The decision can use the whole message state

`attempts so far` `last reply` `TLS outcome` `sender` `verdict at reception` `queue lifetime`

Attempt 1

STARTTLS fails at `mx.partner.example`.

Before attempt 2

The routing service sees the TLS failure of the last attempt.

Attempt 2

The message goes through the VPN relay instead.

Example consumer

DKIM2 needs a hook for each copy and each DSN

1

Each copy transmitted

Signed with the exact envelope of the SMTP transaction that carries it, so blind copies stay out of other copies' signatures.

`transmit stage`

2

Each DSN the MTA generates

Carries its own DKIM2 signature, so it is processed before it is sent.

`dsn stage`

DKIM2 is an example consumer, not a dependency: nothing in the deliverables requires it.

draft-ietf-dkim-dkim2-spec

Demand

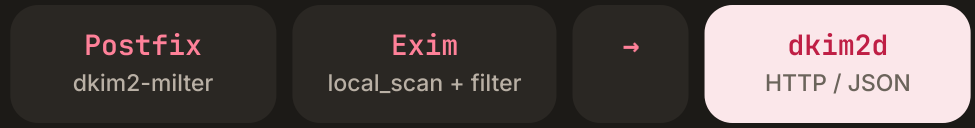
Operators and scanner authors are asking for this

Operators

Site-specific rules: check the recipient against the directory, add the footer legal asked for, send an event to the SIEM.

Stalwart ships Milter support; its users asked for an HTTP interface as well.

dkim2d, an independent DKIM2 implementation



The logic is one HTTP service; each MTA has its own adapter.

The operators of mailbox.org (Heinlein Group) sent detailed proposals that shaped -02.

Design goals

Five goals shape the design

Goal 1

Adopted formats

HTTP, JSON or CBOR, JSON Patch and the JMAP Email model, so existing libraries do most of the work of a targeted filter service.

Goal 2

Negotiated capabilities

A scanner learns at registration what it will receive and may change, and gets explicit errors, never silent absence. Extensions use negotiated namespaces.

Goal 3

Operator-bounded authority

The operator grants each scanner what it may change; by default, only toward stricter processing.

Goal 4

Explicit failure semantics

A time budget on every request, and a chosen behavior when a scanner fails.

Goal 5

Stable identifiers

One queue identifier correlates a message across every stage; a request id stays the same on every retry.

Core idea

One state object, one JSON Patch, and the MTA controls change

1

One state object

At each stage: the session, the envelope with each recipient's status, the message and the delivery attempts.

2

One JSON Patch

The scanner answers with an RFC 6902 patch against that object. The MTA acts on the resulting state.

3

The MTA controls change

Each scanner may change only the paths it was granted.

Accepting, rejecting, adding a header, discarding a recipient, choosing a route and suppressing a DSN are all changes to the state. **Every effect is a path, and every permission is a grant on a path.**

Discovery and registration

The MTA is always the client

Optional

Discover

GET `/.well-known/mta-hooks`

The stages, properties and grants the scanner wants.

Once

Register

POST label and subscriptions. The scanner may narrow them, never widen.

Per stage

Hook request

POST the current state to the `hookEndpoint`.

Per stage

JSON Patch

Validated against the grants, then applied.

No endpoint on the MTA

- No callbacks, no redirects followed
- HTTP/1.1 required, HTTP/2 recommended
- Errors are RFC 9457 Problem Details

Authenticated, over TLS

- Bearer tokens mandatory to implement
- TLS client certificates optional
- TLS 1.2 or later, scanner certificate validated

Subscriptions

The first matching subscription applies

POST /v1/registrations

```
{"name": "mx1.example.com", "label": "spam",
 "softwareVersion": "2.1.0", "timeoutMs": 5000,
 "serialization": "application/json",
 "subscriptions": [
  {"stages": ["data"],
   "filter": {"authenticated": false},
   "properties": ["/session", "/envelope", "/message"],
   "grants": [{"path": "/action"}, {"path": "/response"}],
   "onFailure": "continue"},
  {"stages": ["route"],
   "filter": {"envelopeTo": "*@partner.example"},
   "properties": ["/envelope", "/queue"],
   "grants": [{"path": "/envelope/to/*/route/name",
               "values": ["vpn-partner"]}],
   "onFailure": "tempfail"}]}
```

First match wins

Per stage, the first subscription that lists it and whose filter matches. Each has its own properties, grants and `onFailure`.

`envelopeTo` `clientIp` `authenticated`

`+ JMAP Email filters`

Per recipient, outbound

At `route` and `delivery`, each subscription gets only the recipients it matches. `transmit` and `dsn` match as a whole.

Stages

Nine stages follow a message from connect to DSN

`connect`	`ehlo`	`mail`	`rcpt`	`data`	`route`	`transmit`	`delivery`	`dsn`
before greeting	each EHLO	each MAIL	each RCPT	end of DATA	each attempt	before sending	after the job	before a DSN

Inbound: an action and an SMTP reply

Outbound: the status of each recipient

State accumulates

A scanner subscribed only to `data` gets connection, TLS, authentication, envelope and message in one request, and can still reject at SMTP time.

One queue identifier

The same from `data` to the last `delivery` or `dsn`, so a scanner can keep what it learned at reception.

Hook request

The hook request carries the state the MTA holds

At data, message omitted


```
{
  "stage": "data", "action": "accept", "decidedBy": "mta",
  "response": {"code": 250, "message": "Queued as 4F2A31B0C9"},
  "timestamp": "2026-10-28T16:30:00Z", "timeoutMs": 4200,
  "session": {"id": "01JBDX3TQ8V0D6H1E2F3G4H5J6",
              "startedAt": "2026-10-28T16:29:58Z",
              "client": {"ip": "192.0.2.100", "port": 54321},
              "counters": {"commands": 5, "pregreetBytes": 0}},
  "envelope": {
    "from": {"address": "sender@example.org", "parameters": {}},
    "to": [{"address": "bob@example.com", "parameters": {},
            "status": "accepted", "decidedBy": "mta"}]},
  "senderAuth": {"spfMailFrom": "pass", "dmarc": "pass"},
  "queue": {"id": "4F2A31B0C9"}
}
```


Only what is subscribed

Always the stage, timestamp and time budget; inbound, the action and reply.

The message as a JMAP Email object, as raw bytes, or both. Outbound adds attempts and routes.

HTTP header fields

- `MTA-Hooks-Registration`
- `MTA-Hooks-Request-Id`
- `MTA-Hooks-Context`

Hook response

The answer is a JSON Patch, validated as a whole

Reject a message

```
[
  {"op": "replace", "path": "/action", "value": "reject"},
  {"op": "replace", "path": "/response",
   "value": {"code": 550, "enhancedCode": "5.7.1",
             "message": "Virus detected"}}
]
```

Or nothing

`204`, or an empty patch, means no changes.

The patch applies to the state the MTA holds, so a header can be added without receiving the message.

1

It applies per RFC 6902, including every `test`.

2

Every location that differs is covered by a grant.

3

Ordered properties only move toward strict.

4

Types, stage rules, headers and MIME are valid.

The MTA compares the original and resulting states. All or nothing: an invalid patch is discarded whole.

Grants

Grants bound every change, and restrict by default

```
[
  {"path": "/message/headers"},
  {"path": "/action", "relax": true},
  {"path": "/envelope/to/*/route/name",
   "values": ["default", "vpn-*"]},
  {"path": "/response/delayMs", "max": 30000},
  {"path": "/ext/org.example.spam", "max": 4096}
]
```

- **path**: with `*` for any index
- **values**: allowed strings, with wildcards
- **max**: a number ceiling, or a size in octets
- **relax**: may loosen ordered properties

No grant, no change. The most specific path applies.

Ordered property	Permissive to strict
`/action`	accept, quarantine, then reject, discard, disconnect
`/envelope/to/*/route/tls`	none, opportunistic, required, verified

With relax, the scanner is the final authority for that property: it can override a rejection or lower TLS. Granted explicitly; REQUIRETLS still wins.

If several scanners relax the same property, the last in the chain prevails.

Outcomes

Inbound stages set an action, outbound stages set recipient status

Inbound: /action and /response

Action	Effect
accept, quarantine	Continue, or accept and hold
reject	Send a 4xx or 5xx reply
discard	Accept and drop
disconnect	Send a 421 reply and close

Outbound: /envelope/to/*/status

Stage	From	To
route, transmit, delivery	accepted, deferred	deferred, failed, discarded
delivery	failed	discarded
dsn	accepted	discarded

route

A route per recipient: a configured name, or next hop, port, source IP and TLS. Different routes never share a transaction.

transmit

Per SMTP transaction, with its final envelope. Message changes apply to this copy only; `dsn` works the same way.

Failure and chains

Failures, time budgets and chains are explicit

onFailure: continue

The default. Proceed as if the scanner made no changes.

onFailure: tempfail

Inbound, a 451 or 421 reply. At route and transmit, recipients stay deferred; at dsn, the DSN waits.

Time budget

Each request carries its `timeoutMs`. Retries reuse the same request id.

Chains

Scanners run in configured order, each seeing the state the previous ones left and `decidedBy`. Reject, discard and disconnect end the chain.

Results passed along the chain

```
"ext": {"org.example.spam": {"score": 0.4}}
```

Kept with the message, read again at route, delivery and dsn.

A timeout, an error status or an invalid patch is a failure; the state is left unchanged.

Next: Carsten and Manu

Implementer proposals shaped –02

Routing

The `route` stage, per-recipient routes and TLS policy.

Bounded authority

The operator defines the space; the scanner decides within it.

Observability

Stable queue ids, per-attempt results, time budgets, trace ids.

Inbound signals

Session counters, proxy data, certificates, delayed replies.

Extension data

Namespaced results shared between scanners.

Per-copy changes

Changes to each transmitted copy, at the `transmit` stage.

Carsten Rosenberg and Manu Zurmühl, Heinlein Group (mailbox.org)

The details of their proposals, from operating mail at scale.